The Market Keeps Its Own Clock

Time changes, the vol crush as a reading error, and estimating the clock from the kinks

Vol-Fitter Technical Notes, No. 11

Vol-Fitter

July 27, 2026

Abstract

Implied volatility divides a price-determined quantity — total variance — by a clock, and the clock is a choice. This lecture develops the event variance clock from the theorem that licenses it: a continuous martingale is a Brownian motion run on its own intrinsic clock (Dambis–Dubins–Schwarz), so a scheduled event — earnings, a macro print — is not a new model but a known *acceleration* of that clock, and the overnight vol crush is a reading error: the price and its total variance do not move, only the volatility read against the wrong (calendar) clock does. The production clock adds N_e *extra equivalent days* per event; everything price-like is invariant under the relabeling (quotes, total variance, butterfly and calendar admissibility, the ordering of expiries), while everything per-unit-time is covariant (implied vol drops by exactly $\sqrt{T/\tau}$ — 36 vol bp on the worked event; forward variance transforms by the interval's clock rate). Reading the clock *off* prices is then an inverse problem with sharp limits, all measured here: the dense term-structure curve must be interpolated linearly in τ, not T (the calendar-clock interpolation smears the worked event into a 29-vol-bp artifact between quoted expiries); the auto-calibrator recovers a planted event to 0.10 extra days at single-name vol but is blind below its materiality threshold at index vol (a deliberate sparsity prior), and a flat term structure yields exactly no events. Run on the live AAPL term structure it installs 2 events worth 11.8 extra days — most of them against the earnings-bearing interval — flattening the forward-variance ladder's spread from 127 to 106 variance bp. With an empty calendar the clock is the identity and the entire pipeline is byte-identical.

Contents

1 The wrong clock

A diffusion accrues variance at a steady rate: $w = \sigma^2 T$, linear in time. Real markets refuse the schedule around scheduled events — one earnings day carries the variance of several ordinary days — and a fitter that insists on the calendar clock produces the familiar pathology: a short-dated implied vol that looks "too high," a term structure with a kink no smooth model can represent without distorting the smile, and pre/post-event surfaces that refuse to line up across names and dates. The instinct to resist is enriching the *model* (jumps, event vol processes); the design here is older and cheaper: change the *clock*.

Heuristic

Separate the calendar (how many days to expiry) from the variance budget (how much variance those days carry). An earnings day is one calendar day but, say, five days' worth of variance. Measured in *variance days* the diffusion looks steady again — the "too-high" short-dated vol is just the same total variance divided by a smaller time. The event clock is the change of variable that restores $w = \sigma^2 \tau$ with constant σ.

Invariant

1. **Price preservation.** The clock never touches $w(k)$; it changes only the volatility read-off $\sqrt{w/\tau}$. Adding an event cannot change an option price, hence cannot introduce arbitrage.
2. With an empty calendar, $\tau = T$ exactly and the entire downstream pipeline is byte-identical.
3. Every consumer — every fit, the local-vol mesh, the term structure, the option table — reads the *same* τ: one variance clock per node, not one per view.
4. Editing a calendar refits only the affected ticker (per-ticker events version).

Conventions and the notation ledger. T is calendar maturity in years; $\tau(T)$ is variance time; both are clocks, nothing else is. Subscripted ∂ for derivatives; no primes. A variance bp is 10^{-4} of total variance (or of a forward-variance rate); a vol bp is 10^{-4} of volatility.

$T,\ \tau(T)$	calendar / variance years	$t_e,\ N_e$	event date; extra equivalent days
$w(k)$	total implied variance (price-fixed)	$\sigma_{\text{work}} = \sqrt{w/\tau}$	working (clock) vol
$f_i = \Delta w_i / \Delta \tau_i$	forward variance rate	$M,\ Z,\ \langle M \rangle$	martingale; Brownian; quadratic variation

Table 1: Every symbol in the note. N_e is always *days*, never years — the unit trap of remark 1.

2 The license: martingales carry their own clock

Theorem 1 (Dambis; Dubins–Schwarz [1, 2])**.** *Let M be a continuous local martingale with $M_0 = 0$ and $\langle M \rangle_\infty = \infty$. Then there is a standard Brownian motion Z with*

$$M_T = Z_{\langle M \rangle_T} \qquad \text{for all } T.$$

Every continuous martingale — in particular the normalized forward price of Note 10's unnamed dynamics — *is* Brownian motion, run on the clock of its own accumulated variance. The market

keeps that clock; the calendar is ours. Implied volatility is what appears when the market's motion is read against the calendar: smooth stretches read as ordinary vol, and a scheduled burst of $\langle M \rangle$ — an event — reads as a short-dated vol spike, though nothing about the motion's law at expiry has changed. The variance clock is simply a deterministic *forecast* of $\langle M \rangle$: ordinary days advance it at rate one, scheduled events add mass.

2.1 The clock, and what a clock can never change

Each event contributes N_e *extra* equivalent days ($N_e = 4$ means the day counts as five). The variance time of maturity T is

$$\boxed{\tau_{\text{days}}(T) = 365\,T + \sum_{t_e \le T} N_e, \qquad \tau(T) = \frac{\tau_{\text{days}}(T)}{365},} \tag{1}$$

counting only events at or before T. Figure 1A draws it: the calendar line, a jump of $N_e/365$ at the event, parallel thereafter. With no events, $\tau(T) = T$ exactly.

Proposition 1 (Invariance and covariance under a change of clock)**.** *Let τ be any strictly increasing relabeling of maturity with $\tau(0) = 0$. Then: (i) option prices, and hence total variances $w(k, T)$, are invariant — the relabeling attaches the same terminal laws to the same expiries; (ii) butterfly admissibility per expiry and calendar order across expiries are invariant — Note 10's four faces compare the same objects in the same order, since τ is monotone; (iii) implied volatility is covariant: $\sigma_{\text{work}} = \sqrt{w/\tau} = \sigma_{\text{cal}}\sqrt{T/\tau}$; (iv) forward variance is covariant by the interval clock rate: $\Delta w/\Delta\tau = (\Delta w/\Delta T) \cdot (\Delta T/\Delta\tau)$.*

Proof. By inspection: a deterministic relabeling changes no random variable and no expectation, only the number the same total variance is divided by. (ii) is the monotonicity remark of Note 10's assumption box. □

The proposition is the note's entire safety case. Everything a desk can trade is in the invariant column; everything the clock changes is a *reading*. In particular the invariant box's price preservation is not a property the implementation must defend — it is the arithmetic fact that the clock enters nowhere except as a denominator.

2.2 The crush, quantified

Insert an event of N_e extra days before a fixed expiry T. Total variance is pinned by the price, so by proposition 1(iii) the working vol drops by the factor $\sqrt{T/\tau}$:

$$\sigma_{\text{work}} = \sigma_{\text{cal}} \sqrt{\frac{T}{T + N_e/365}}. \tag{2}$$

On the worked event (four extra days at $t_e = 0.25$, probe expiry just past it) the drop is 36 vol bp at a 20% vol — Exercise 1 checks the arithmetic. Figure 1B shows the term-structure view: against the variance clock the vol curve is flat by construction; read against the calendar, the same prices display the familiar event hump — lifted before expiry, decaying as the event becomes a smaller fraction of the maturity. This *is* the vol crush: after the event resolves and its N_e leaves

the calendar, the calendar reading falls to the flat clock vol, while nothing about the option prices jumped at all.

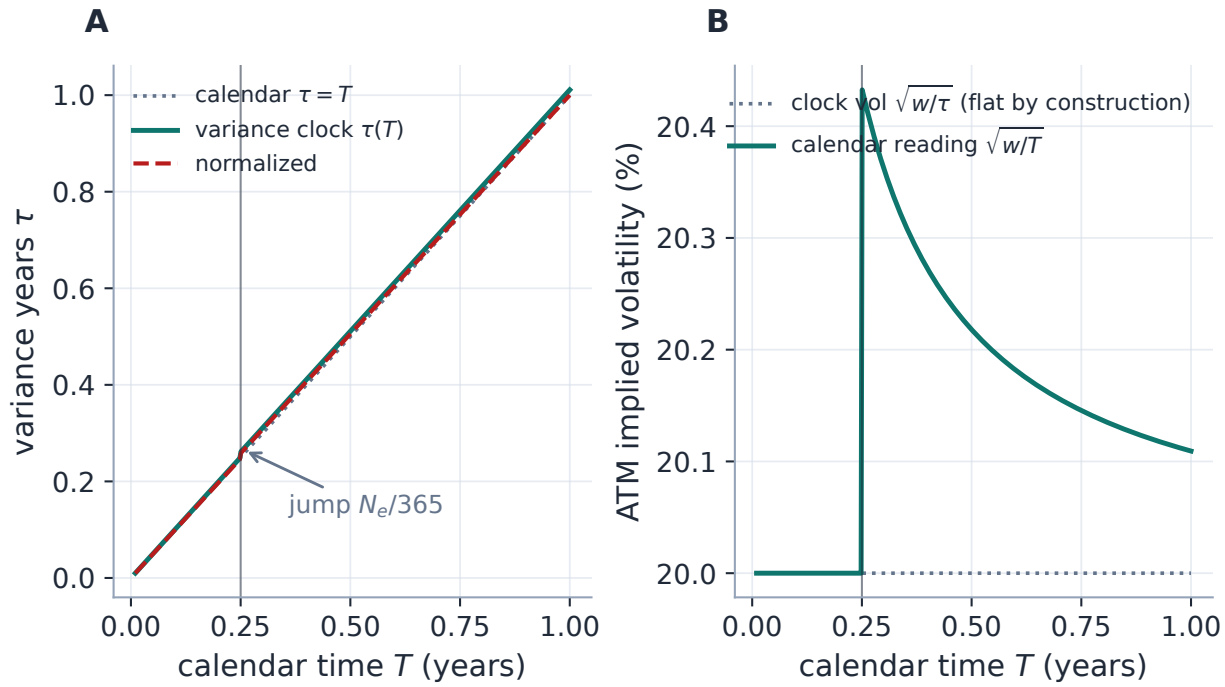

Figure 1: Two clocks, one set of prices (production clock code). A: the variance clock (1) jumps $N_e/365$ at the event; the normalized variant (dashed) rescales so the one-year budget is unchanged. B: the same total-variance curve read two ways — flat against its own clock, the familiar event hump against the calendar. The hump is a property of the ruler, not of the prices.

The clock is a few lines, verified against production to 1.0×10^{-17} (appendix C):

Listing 1: The event variance clock (1) (distilled from `calib/weighted_time.py`).

```
def weighted_variance_years(t_cal, events, normalize=False, dpy=365.0):
    if t_cal <= 0: return t_cal          # events: (t_e, N_e), N_e = EXTRA days
    tau_days = t_cal*dpy + sum(N for te, N in events if te <= t_cal and N > 0)
    if normalize:                        # rescale so a 1Y budget stays dpy days
        e1 = sum(N for te, N in events if te <= 1.0 and N > 0)
        tau_days *= dpy/(dpy + e1)
    return tau_days/dpy                  # tau(T); equals t_cal with no events
```

2.3 Normalization: fixing the year's budget

Un-normalized, the cumulative weight exceeds the day count ($\tau > T$). The normalization toggle rescales *all* days by one factor so the one-year budget stays 365:

$$\tau_{\text{days}}(T) \;\longmapsto\; \tau_{\text{days}}(T) \cdot \frac{365}{365 + \sum_{t_e \le 1} N_e}. \tag{3}$$

Then the one-year variance time — and the one-year implied vol — match a no-event year exactly: events *redistribute* variance within the year rather than add it. Which convention to run is a desk choice, not a mathematical one: un-normalized treats events as genuinely extra variance against a one-per-day baseline; normalized treats the year's budget as known and the events as its scheduled lumps. Exercise 3 computes what normalization does to the sub-year readings.

Exercise 1. Derive (2) from proposition 1(iii) and evaluate it for the worked event: $T = 0.30$, $N_e = 4$, so $\tau = 0.3110$. Confirm the 36-vol-bp drop at $\sigma_{\text{cal}} = 20\%$, and note the crush scales like $N_e/(2 \cdot 365\,T)$ for small events — steepest for the shortest expiries, exactly where desks watch it.

3 Reading the clock off prices

Everything so far ran the clock forward: given a calendar, re-read the surface. The productive direction is backwards: the market's prices carry information about the clock, and two production mechanisms extract it.

3.1 Interpolate in the clock, not in the calendar

Between quoted expiries the term view needs a dense curve. The production rule interpolates total variance *linearly in* τ — constant forward variance per unit of the market's clock, the discrete restatement of "Brownian on its own clock" — with rate-preserving extrapolation at both ends. Figure 2 is the wrong-way demo: on quotes generated by a flat clock vol around one event, the production interpolation reproduces the flat-then-hump reading exactly, while the calendar-clock interpolation smears the event's variance across the whole bracketing interval — a 29-vol-bp artifact at expiries nobody quotes, largest just before the event where a desk would price an event-straddle off the dense curve.

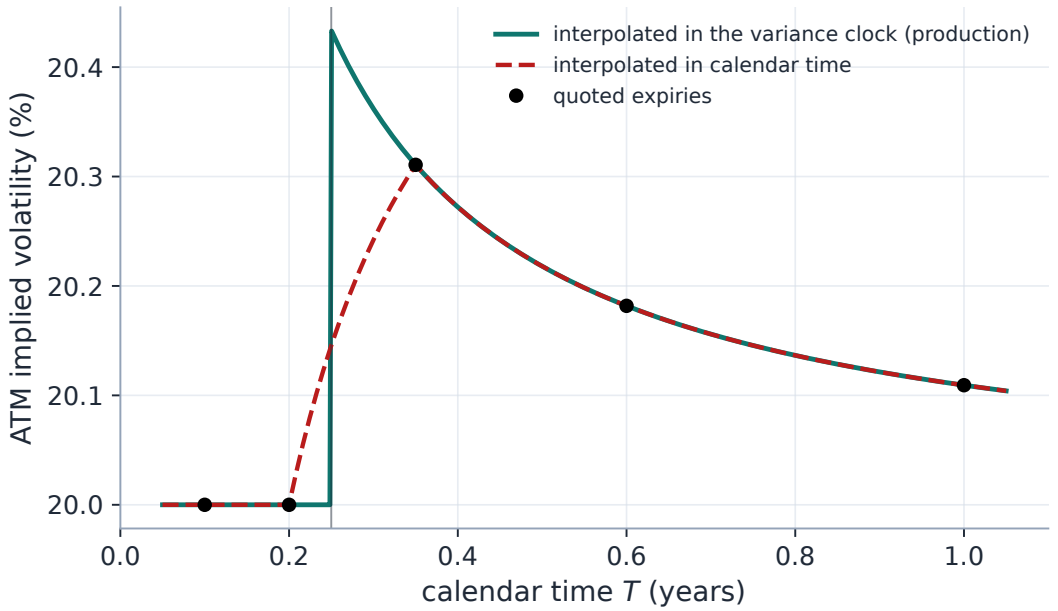

Figure 2: Interpolate in the clock, not the calendar (production interpolator both ways). Quotes (dots) are generated by a flat vol on the event clock. Linear-in-τ interpolation (teal) reproduces the true reading: flat to the event, the jump, the decay. Linear-in-T interpolation (dashed) smears the event's variance across the bracketing interval — up to 29 vol bp of phantom vol at unquoted maturities. The event sits *between* quotes; only the clock knows it is there.

3.2 The auto-calibrator: an inverse problem with a sparsity prior

The calendar need not be typed in. The term workspace's auto-calibrate action infers it from the ATM term structure: place one candidate event before each quoted expiry up to a chosen horizon, and solve for non-negative extra days $\{N_i\}$ minimizing

$$J(N) = \sum_i \left(f_{i+1} - f_i\right)^2 + \lambda_{\text{mono}} \sum_i \left(\min(f_{i+1} - f_i, 0)\right)^2 + \lambda_1 \sum_i \frac{N_i}{365} + \lambda_2 \sum_i \left(\frac{N_i}{365}\right)^2, \qquad (4)$$

where $f_i = \Delta w_i / \Delta\tau_i(N)$ is the *event-time* forward variance of interval i (a bounded quasi-Newton solve; events under half a day are thresholded away; the first interval past the horizon carries no candidate and anchors the tail). Three structural facts make (4) sound. The calendar-time forward variance is event-*invariant* — prices fix w (proposition 1) — so the objective must work on the dilated clock, the only one events move. An event can only *lengthen* its interval's weighted time, so it can only pull a forward-variance spike *down* toward its neighbours: the solver reads the calendar off exactly the kinks the clock was built to explain. And the asymmetric monotonicity term spends its budget on *decreases* of forward variance — the shape a genuine pre-event interval produces — rather than on an ordinary upward-sloping term structure.

What can such an estimator promise? Figure 3 is the audit. At single-name vol (40%) a planted event is recovered at the correct interval to within 0.10 extra days across sizes — the small downward bias is the ℓ_1 term's shrinkage, the standard price of a sparsity prior. At index vol (20%) the same planted events sit *below the solver's materiality threshold* up to 5 extra days: the flatness gain a small event offers scales with σ^4 while the sparsity charge does not, so quiet names keep empty

calendars unless the kink is large — a deliberate design, not a failure. A flat term structure returns exactly 0 events.

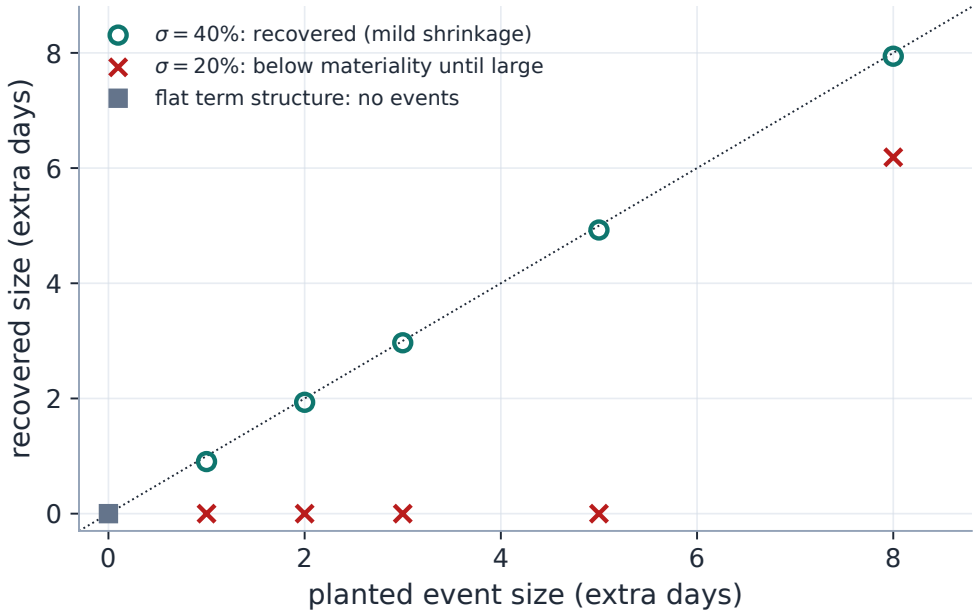

Figure 3: What the inverse problem can promise (production solver). Planted events are recovered on the diagonal at single-name vol, with the ℓ_1 shrinkage visible as a downward bias of at most 0.10 days; at index vol the same events fall below the materiality threshold until they are large — the sparsity prior working as designed. A flat term structure solves to an empty calendar.

3.3 Reading a live clock: AAPL

Figure 4 runs the production solver on the real AAPL term structure of the standing export (4 expiries, valuation date 2026-07-18). Per *calendar* year, the forward-variance ladder is uneven — the interval containing the late-October earnings runs visibly hot. The solver installs 2 events worth 11.8 extra days in total, the bulk against the earnings-bearing interval, and the ladder's spread tightens from 127 to 106 variance bp. Two honest readings of that exhibit. The solver has recovered, from prices alone, that this interval contains materially more than its share of calendar days' variance — the market's clock runs fast through it. But the solver cannot *name* the reason: it reads kinks, and every source of elevation — earnings, a macro date, genuine term-structure shape — looks identical through (4). The installed calendar is an estimate to be reviewed and edited in the term workspace, not an oracle (section 6).

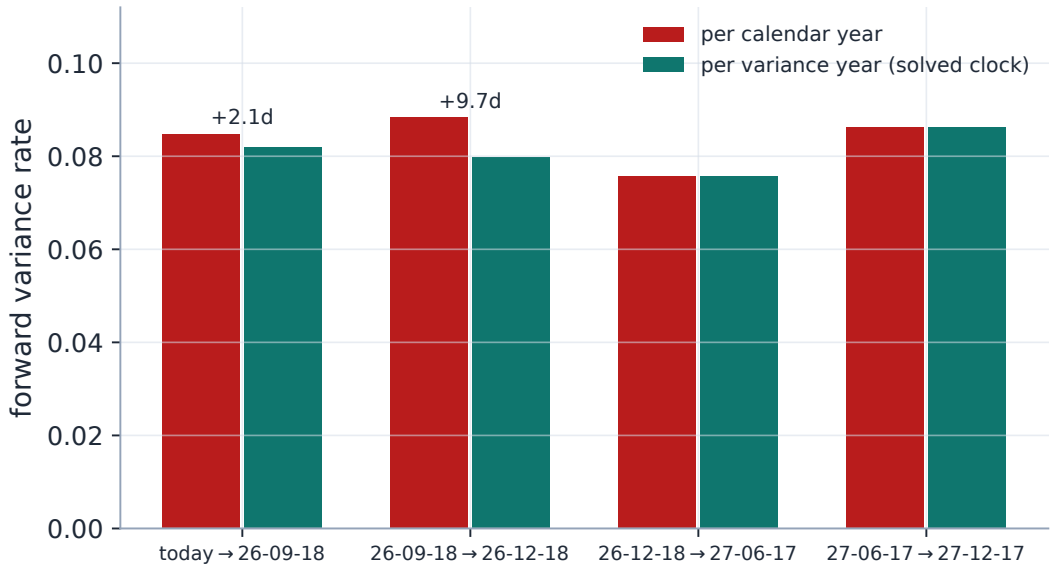

Figure 4: Reading the clock off live AAPL (production solver on the standing export's term structure). Per calendar year (rust) the earnings-bearing interval runs hot; the solver installs 11.8 extra days (annotated), and per variance year (teal) the ladder flattens — forward-variance spread 127 to 106 variance bp. The market's clock runs fast through that interval; the solver measures by how much, not why.

Exercise 2. Prove the within-interval non-identifiability: the fits and every consumer read the clock only through τ at the quoted maturities (1), which depends on the calendar only through the cumulative sums $\sum_{t_e \le T_i} N_e$. Conclude that two events inside the same quoted interval are indistinguishable from one event of their combined size anywhere in that interval — which is why the solver's midpoint placement is a convention, not an estimate — and that an event's *date* becomes identified only when an expiry is listed between it and its neighbours.

Exercise 3. With normalization on (3) and the worked calendar (one event, $N_e = 4$, $t_e = 0.25$), compute the working vol of a six-month expiry relative to the un-normalized clock, and verify the one-year reading is exactly the no-event one. Then say in one sentence why normalization cannot affect the auto-calibrator's solved events (hint: one global factor cancels in every forward-variance ratio the flatness term compares).

4 One clock through the whole pipeline

The clock is computed once per node — τ is stored beside the calendar maturity in the prepared quotes, and folded into the fit cache key — and every consumer reads that one number: the parametric and local-vol fits, the vol↔variance conversions (including Note 08's var-swap targets), the term structure, the option table, and the 3-D surface mesh, which is quoted in $\sqrt{w/\tau}$ and carries both clocks so a stacked view can recover price variance (invariant 3; a surface tab reading the calendar clock while the smile read the variance clock was a real, test-locked regression). Editing a calendar bumps a per-ticker events version, so one name's earnings entry never refits the rest of the

universe (invariant 4); an empty calendar short-circuits to $\tau = T$ and the pipeline is byte-identical (invariant 2).

4.1 Below one day: the session clock

The thesis of this note does not stop at the day boundary: below one day, the market's clock is not calendar-proportional either, and the machinery for reading it is shipped, not hypothetical. With the intraday clock enabled (a surfaced toggle, off by default and byte-identical off), two things change. First, *where maturity ends*: each node is valued from the chain snapshot's timestamp to the expiry's exact *settlement instant* — the stored settlement map, with exchange session rules as fallback — so a same-day expiry prices over its remaining hours rather than an unrepresentable zero of calendar days. Second, *how a day accrues*: variance time flows through a session-weighted profile in which a fraction of each trading day's variance (default 6.5/24) accrues during the exchange session and a non-trading calendar day carries weight one. Those two defaults are not a claim about the physics; they are a *nesting* property, chosen so that any close-to-close span of N calendar days integrates to exactly N day-weights — the legacy convention recovered to the day, so switching the clock on changes sub-day reads and nothing else. The physics lives in the research settings: session shares near 0.7–0.9 make a live same-day expiry's clock "remaining trading minutes" and the overnight cheap.

The measured case for a non-proportional sub-day clock is the weekend. Across the stored intraday filter campaign, a thirty-minute session step moves ATM volatility about 19.5 bp — while one overnight and an entire three-day weekend *both* move it about 55 bp. No clock proportional to elapsed calendar time can calibrate all three numbers at once; a session-weighted one can. Two honest scope statements complete the picture. The in-session profile is uniform in this version — the U-shaped open/close seasonality is a documented follow-up, deliberately not yet a knob. And the elegant part costs nothing: an event's date is already stored as a year fraction, so once the clock's base is sub-day, a *fractional* event date composes with (1) unchanged — an 08:30 CPI print becomes a genuine intraday event with no new event machinery at all.

Remark 1 (One production clock, one legacy clock, one unit trap)**.** Two dilation clocks exist in the codebase and must not be conflated. The production clock is (1) — day-weighted, the one every fit and view consumes, including the production term-structure interpolator of section 3. The older `EventClock` survives only in its own tests: it predates the day convention (its weights are *years* of equivalent diffusion time) and its interpolation method is the historical ancestor of the shipped one. Hence the unit trap: an event's `weight` is *extra equivalent days*, never years — the schema docstring said years until 2026-07-09 and the legacy module's comments remain the stale party. When in doubt, the production semantics is the one listing 1 and its tests lock.

Remark 2 (The other clock in the building)**.** The observation filter of Note 15 carries its own *session clock* for a different question entirely: how much handle drift to expect *between two snapshots* (a weekend moves ATM about as much as one overnight). That clock budgets process noise between fits; this note's clock dilates maturity within one fit. They share the intraday primitive and nothing else — separate toggles, separate subsystems, different default shares tuned for different properties (the filter's share is 0.60 with closed days at weight zero; this note's is the nesting pair of section 4.1), and no interaction.

5 What is genuinely original here

Time changes are classical (Bergomi dilates around events; Ané and Geman read equity returns as Brownian on a transaction clock). The contributions are the engineering shape. *Price preservation by construction*: the clock enters only as a denominator, so the invariance column of proposition 1 is arithmetic, not a property to defend — adding an event can never move a price or create arbitrage. *The day-weighted parametrization*: one intuitive number per event (extra equivalent days) a desk can quote, with the normalization (3) cleanly separating "extra variance" from "reallocated budget." *The solver as a disciplined inverse*: flatten event-time forward variance under a sparsity prior, with the identifiability limits measured rather than hidden (fig. 3) — shrinkage quantified, materiality threshold explicit, flat input provably eventless. And the byte-identical empty calendar makes the whole feature strictly additive. The payoff is comparability: pre- and post-earnings surfaces line up across names and dates because event-time vol is the signal — calendar-time vol is the artifact.

6 Limitations

Where the guarantees stop. *The clock is deterministic*: it forecasts the schedule of $\langle M \rangle$, not its randomness — no vol-of-vol, no stochastic event size; that is model territory (Bergomi [4]), deliberately out of scope. *Events must be scheduled*: an unscheduled jump is not a clock feature, and the crush factor (2) prices only the anticipated part. *The solver reads ATM only*: the auto-calibrator consumes the ATM total-variance ladder; the smile's event signature (strangle richness) is unused information. *Placement is a convention*: within a quoted interval an event's date is unidentified (Exercise 2), and the midpoint is a choice. *Sizes are shrunk and thresholded*: the ℓ_1 prior biases solved days downward by up to 0.10 on the audit and zeroes sub-materiality events entirely on quiet names (fig. 3) — review the installed calendar rather than trusting it. *Kinks have no fingerprint*: the solver cannot distinguish an earnings kink from genuine term-structure shape (the AAPL exhibit's honest reading). And *normalization is a convention*, not an inference: it changes every sub-year reading while leaving prices, fits and solved events untouched.

A Hyperparameter atlas

The only home for settings names: the body speaks mathematics, this table speaks configuration.

Table 2: Event-clock hyperparameters.

Knob	Default	Role
Surfaced		
`eventsEnabled`	`true`	Master toggle: off forces $\tau = T$ everywhere.
`normalizeEvents`	`false`	The one-year budget normalization (3).
`intradayClock`	`false`	Sub-day maturity: settlement- instant valuation plus the session-weighted profile (section 4.1); byte-identical off.

Knob	Default	Role
`sessionVarShare`	6.5/24	In-session share of a trading day's variance; the default is the flat-density value that nests the legacy day convention. Read only while the intraday clock is on.
`nonTradingWeight`	1.0	Day-weight of a non-trading calendar day — the weekend-effect research lever. Read only while the intraday clock is on.
per-ticker calendar	empty	`EventSpec`($t_e > 0$, $N_e \geq 0$, label), N_e in *extra equivalent days*; persisted per ticker (its own endpoint and version), not in the global options.
`maxExpiry`	—	Auto-calibrate horizon (per request): one candidate event per expiry at or before it.
Hidden		
`DAYS_PER_YEAR`	365	Calendar-to-variance day conversion.
`mono_weight`	1.0	Auto-calibrate penalty on *decreasing* event-time forward variance (4).
`sparse_weight` / `ridge_weight`	10^{-3} / 10^{-4}	The ℓ_1/ℓ_2 priors keeping solved events small and sparse.
`min_event_days`	0.5	Sparsity threshold: smaller solved events are dropped.
`max_event_days`	1825	Solver box bound per event.
`base_days`	`None`	Internal carrier of `intradayClock`: the session-profile day-weights that replace the calendar base (section 4.1).

Performance. The clock is a scalar sum per node — negligible; with no events $\tau = T$ and the feature costs exactly nothing. The auto-calibrator is one bounded quasi-Newton solve over a handful of variables, interactive. A calendar edit refits one ticker (per-ticker events version in the fit key). Figures and macros were generated at commit `25e72e5` on 2026-07-19; the session clock's weekend evidence (section 4.1) is quoted from the stored intraday filter campaign, not regenerated here.

B Traceability

Table 3: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor *Test anchor*
No events: identity clock, byte-identical pipeline	invariant 2	`volfit/calib/weighted_time.py` `tests/test_weighted_time.py::test_no_events_is_identity`; `::test_no_event_byte_identical`

Claim	Object	Code anchor *Test anchor*
Session clock: settlement instant, nesting profile, off = byte-identical	section 4.1	`volfit/calib/intraday_time.py`, `volfit/data/expiry_time.py` `tests/test_intraday_time.py`; `tests/test_intraday_0dte.py`; certification case `0dte_exit_gates`
Event adds day weights before its date only	(1)	`volfit/calib/weighted_time.py` `tests/test_weighted_time.py::test_event_before_adds_day_weights`
Price preservation: readings scale by $\sqrt{T/\tau}$, LV included	(2)	`volfit/api/service.py`, `volfit/api/displayed.py` `tests/test_weighted_time.py::test_event_lowers_iv_by_sqrt_t_over_tau`; `::test_localvol_iv_drops_with_event`
Normalization pins the one-year budget	(3)	`volfit/calib/weighted_time.py` `tests/test_weighted_time.py::test_normalization_pins_one_year`; `::test_normalization_keeps_one_year_vol`
Surface mesh shares the smile's clock	section 4	`volfit/api/affine_views.py`, `volfit/api/surface.py` `tests/test_surface_clock.py::test_surface_mesh_uses_variance_clock_like_the_smile`
Dilated interpolation recovers lumped event variance; legacy clock confined to its tests	fig. 2	`volfit/calib/weighted_time.py`, `volfit/calib/event_time.py` `tests/test_event_time.py::test_interpolation_lumps_event_variance`
Auto-calibrate flattens the dilated forward variance; flat input stays eventless; horizon respected; endpoint installs the shared calendar	(4)	`volfit/calib/event_autocalib.py`, `volfit/api/event_autocalib.py` `tests/test_event_autocalib.py::test_spike_is_flattened`; `::test_flat_input_stays_eventless`; `::test_horizon_limits_events`; `::test_autocalibrate_endpoint_sets_calendar`

C Reference implementation

Listing 1 was executed against the production `weighted_variance_years` by this edition's generator on every run — both normalization settings, a four-event calendar, maturities to two years — agreeing to 1.0×10^{-17} (floating-point identity). The production module also carries the dense-curve interpolator used in fig. 2 (linear in τ, rate-preserving beyond the quoted ends); the figure exercises the shipped function on both clocks rather than reimplementing either.

References

[1] K. Dambis. On the decomposition of continuous submartingales. *Theory Probab. Appl.*, 10:401–410, 1965.

[2] L. Dubins and G. Schwarz. On continuous martingales. *Proc. Nat. Acad. Sci.*, 53:913–916, 1965.

[3] T. Ané and H. Geman. Order flow, transaction clock, and normality of asset returns. *J. Finance*, 55(5):2259–2284, 2000.

[4] L. Bergomi. *Stochastic Volatility Modeling*. CRC Press, 2016.

[5] J. Gatheral. *The Volatility Surface*. Wiley, 2006.